

Documentação Técnica: Sistema de Gerenciamento de Biblioteca

Ana Maria Fausta Bonfim Santos - SP316635X
Juliana Xavier dos Santos Araújo - SP3154904
Leticia Cardoso Furtado - SP3166261
Thiago Galvão Ferreira - SP3154084

1. Descrição do Sistema e Arquitetura

1.1. Visão Geral

Este projeto é uma aplicação Web Full-Stack para o gerenciamento de uma biblioteca. O sistema permite o cadastro de livros, gerenciamento de usuários com diferentes níveis de permissão (Comum e Administrador) e o controle de empréstimos e devoluções.

A aplicação é dividida em dois componentes principais: um **Backend RESTful** (servidor) construído com Spring Boot e um **Frontend** (cliente) construído como uma Single Page Application (SPA) em React.

1.2. Stack de Tecnologias

- **Backend:**
 - **Framework:** Spring Boot 3+ (Java 21+)
 - **Persistência:** Spring Data JPA (Hibernate)
 - **Segurança:** Spring Security 6+
 - **Autenticação:** JSON Web Token (JWT)
 - **Banco de Dados:** H2 (para desenvolvimento) / PostgreSQL ou MySQL (para produção)
 - **Documentação:** Springdoc-OpenAPI (Swagger UI)
- **Frontend:**
 - **Biblioteca:** React 18+ (com Hooks)
 - **Build Tool:** Vite
 - **Cliente HTTP:** Axios
 - **Estilização:** CSS Padrão

1.3. Arquitetura

O sistema segue uma arquitetura moderna e desacoplada, composta por um monólito stateless no backend e uma SPA no frontend.

1.3.1. Arquitetura do Backend (Servidor)

O backend é uma API RESTful stateless (não armazena sessão) e segue uma arquitetura em camadas (N-Tier) para garantir a separação de responsabilidades (SoC).

1. **Camada de Controller (API):** (/controller)
2. **Camada de Serviço (Lógica de Negócio):** (/service)
3. **Camada de Repositório (Acesso a Dados):** (/repository)
4. **Camada de Domínio (Entidades):** (/model)

1.3.2. Arquitetura de Segurança (Spring Security)

A segurança é baseada em Token (JWT), garantindo que a API seja stateless.

1. **AuthenticationController:** Endpoint público (/api/auth/login) que recebe credenciais e, se válidas, usa o **TokenService** para gerar um JWT assinado.
2. **SecurityFilter:** Um filtro que intercepta *todas* as outras requisições, valida o JWT enviado no header Authorization e autentica o usuário no contexto do Spring.
3. **SecurityConfig:** Define quais rotas são públicas (ex: /login), quais são protegidas (.authenticated()) e quais exigem papéis específicos (.hasRole("ADMIN")).

1.3.3. Arquitetura do Frontend (Cliente)

O frontend é uma Single Page Application (SPA) baseada em componentes React.

- **App.jsx:** Componente raiz que gerencia o estado global da aplicação, incluindo o token de autenticação e os papéis do usuário (isAdmin).
- **Gerenciamento de Estado:** Utiliza os hooks useState (para dados) e useEffect (para carregar dados da API).
- **Renderização Condicional:** O App.jsx renderiza diferentes "telas" (Login, Dashboard de Usuário, Painel de Admin) com base no estado de autenticação (token) e autorização (isAdmin).
- **Serviço de API:** O axios é usado para todas as comunicações com o backend, interceptando erros e atualizando o estado da UI.

1.4. Detalhamento dos Pacotes e Componentes

Abaixo está uma descrição detalhada de cada arquivo .java no backend, explicando seu papel na arquitetura.

1.4.1. Raiz (com.library.project)

- **ProjectApplication.java:** O ponto de entrada (entry point) da aplicação Spring Boot. Contém o método main e a anotação @SpringBootApplication, que inicializa todo o sistema.

1.4.2. Pacote config (Configuração)

Classes que configuram o comportamento central do Spring.

- **AppConfig.java:** Fornece beans de configuração globais. Atualmente, expõe o PasswordEncoder (usando BCryptPasswordEncoder) como um bean gerenciado pelo Spring, permitindo que ele seja injetado em qualquer serviço que precise codificar ou verificar senhas.
- **DataLoader.java:** Uma classe de setup de desenvolvimento. Implementa CommandLineRunner para executar um bloco de código *após* a aplicação iniciar. É responsável por popular o banco H2 em memória com dados de teste (usuários "admin" e "aluno", papéis "ROLE_ADMIN" e "ROLE_COMUM", e livros de exemplo).
- **OpenApiConfig.java:** Configura a documentação do Swagger (OpenAPI). Define o esquema de segurança global (@SecurityScheme) como "bearerAuth" (JWT), permitindo que o Swagger UI exiba o botão "Authorize" e saiba como enviar o token.
- **SecurityConfig.java:** O cérebro da segurança.
 1. Define o SecurityFilterChain (a cadeia de filtros de segurança).
 2. Configura a API para ser stateless (sem sessão).
 3. Define as regras de autorização (authorizeHttpRequests), especificando quais rotas são públicas (.permitAll()), quais exigem autenticação (.authenticated()) e quais exigem papéis específicos (.hasRole("ADMIN")).
 4. Expõe o AuthenticationManager como um Bean.
 5. Registra o SecurityFilter customizado no pipeline.
 6. Configura o CorsConfigurationSource para permitir requisições do frontend React (ex: localhost:5173).

1.4.3. Pacote controller (Camada de API)

Classes anotadas com @RestController que lidam com requisições HTTP e respostas JSON.

- **AdminController.java:** Expõe endpoints restritos a ROLE_ADMIN (/api/admin/**). É responsável pelo CRUD de Usuários (listar, criar, atualizar, deletar) e pela exibição do dashboard de estatísticas do admin.
- **AuthenticationController.java:** Expõe o endpoint público /api/auth/login. Recebe um LoginDTO, valida as credenciais usando o AuthenticationManager e retorna um LoginResponseDTO (contendo o token, nome e papéis).
- **EmprestimoController.java:** Expõe endpoints para usuários autenticados (/api/emprestimos/**). Lida com as ações de realizarEmprestimo e realizarDevolucao, usando @AuthenticationPrincipal para identificar o usuário

logado.

- **GlobalExceptionHandler.java:** Um `@ControllerAdvice` que atua como um "para-raios" para toda a aplicação. Intercepta exceções (`RuntimeException`, `MethodArgumentNotValidException`) e as formata em respostas JSON padronizadas, evitando que o usuário receba erros técnicos (como stack traces).
- **LivroController.java:** Expõe o CRUD de Livros (`/api/livros/**`) e a busca (`/search`). Os métodos de escrita (POST, PUT, DELETE) são protegidos para `ROLE_ADMIN`, enquanto a busca é aberta para qualquer usuário autenticado.
- **ProfileController.java:** Expõe endpoints para o usuário gerenciar *seu próprio* perfil (`/api/profile/**`). Inclui a mudança de senha (`/change-password`) e a visualização do dashboard pessoal (`/dashboard`).

1.4.4. Pacote dto (Data Transfer Objects)

POJOs (ou Records) que definem os "contratos" de dados da API, separando a lógica interna (Entidades) da externa (JSON).

- `AdminDashboardStatsDTO`: Define os dados para o dashboard do admin (total de usuários, livros, etc.).
- `AdminUsuarioCreateDTO`: JSON para um *admin* criar um novo usuário (permite definir papéis).
- `AdminUsuarioUpdateDTO`: JSON para um *admin* atualizar um usuário (nome e papéis).
- `EmprestimoCreateDTO`: JSON para criar um empréstimo (contém apenas `livroId`).
- `EmprestimoDTO`: JSON de resposta para um empréstimo (contém nomes e datas formatadas).
- `LivroComStatusDTO`: DTO de resposta da busca, contendo dados do livro + `emprestimoId` do usuário logado.
- `LivroDTO`: DTO padrão para criar (POST) ou atualizar (PUT) um livro.
- `LoginDTO`: JSON de requisição para login (username, password).
- `LoginResponseDTO`: JSON de resposta do login (token, nomeCompleto, roles).
- `PasswordChangeDTO`: JSON para o usuário mudar a própria senha (`currentPassword`, `newPassword`).
- `TokenDTO`: (Obsoleto, substituído por `LoginResponseDTO`).
- `UserDashboardDTO`: JSON de resposta para o dashboard do usuário (lista de `EmprestimoDTO`).
- `UsuarioCreateDTO`: JSON para registro público (não implementado, mas DTO existe).
- `UsuarioDTO`: DTO de resposta padrão para um usuário (esconde a senha).

1.4.5. Pacote model (Camada de Domínio)

As Entidades JPA que mapeiam o banco de dados.

- **Emprestimo.java:** Entidade que representa um empréstimo. Possui relacionamentos ManyToOne com Usuario e Livro.
- **Livro.java:** Entidade que representa um livro no catálogo.
- **Role.java:** Entidade que representa um papel de usuário (ex: "ROLE_ADMIN").
- **Usuario.java:** Entidade que representa um usuário. Implementa a interface UserDetails do Spring Security, fazendo a ponte entre o JPA e a autenticação.

1.4.6. Pacote repository (Camada de Persistência)

Interfaces que estendem JpaRepository. O Spring Data JPA implementa os métodos em tempo de execução.

- **EmprestimoRepository.java:** Fornece métodos de consulta para a entidade Emprestimo (ex: findByUsuarioAndDataDevolucaoEfetivaIsNull, countByDataDevolucaoEfetivaIsNull).
- **LivroRepository.java:** Fornece métodos de consulta para Livro (ex: findByTituloContainingIgnoreCaseOrAutorContainingIgnoreCase).
- **RoleRepository.java:** Fornece método para buscar um Role pelo nome (findByNome).
- **UsuarioRepository.java:** Fornece métodos de consulta para Usuario (ex: findByUsername, findByUsernameContainingIgnoreCaseAndIdNot).

1.4.7. Pacote security (Segurança)

Classes focadas no mecanismo de autenticação JWT.

- **SecurityFilter.java:** Um OncePerRequestFilter que é executado a cada requisição. Ele extrai o token JWT do header, valida-o (usando o TokenService) e, se for válido, autentica o usuário no SecurityContextHolder.
- **TokenService.java:** Classe utilitária (uma Fábrica) responsável por gerarToken (assinar um JWT com os dados do usuário) e validarToken (verificar a assinatura e expiração).

1.4.8. Pacote service (Camada de Negócio)

Classes anotadas com @Service que contêm a lógica de negócio e orquestram os repositórios.

- **AuthenticationService.java:** Implementa UserDetailsService. Sua única responsabilidade é dizer ao Spring Security *como* carregar um UserDetails (nosso Usuario) a partir do banco de dados durante o processo de login.
- **DashboardService.java:** Atua como uma Fachada (Facade). Coleta dados de múltiplos repositórios (UsuarioRepository, LivroRepository, EmprestimoRepository)

para construir os DTOs de dashboard (AdminDashboardStatsDTO, UserDashboardDTO).

- **EmprestimoService.java:** Lida com as regras de negócio de empréstimos (ex: "chechar se livro tem estoque", "chechar se usuário atingiu limite", "atualizar estoque ao devolver").
- **LivroService.java:** Lida com as regras de negócio de livros (ex: "não excluir livro se tiver empréstimo ativo", "chechar ISBN duplicado", "construir o LivroComStatusDTO").
- **UsuarioService.java:** Lida com as regras de negócio de usuários (ex: "não excluir último admin", "não excluir usuário com empréstimos", "verificar senha atual antes de mudar").

2. Diagramas de Modelo

2.1. Diagrama de Classes (Domínio JPA)

Este diagrama mostra as entidades Java e seus relacionamentos, incluindo a implementação de UserDetails pelo Usuario para integração com o Spring Security.

classDiagram

```
class Usuario {
    +Long id
    +String username
    +String password
    +String nomeCompleto
    +getAuthorities() Collection~GrantedAuthority~
}
class Role {
    +Long id
    +String nome
}
class Livro {
    +Long id
    +String titulo
    +String autor
    +String isbn
    +int quantidadeDisponivel
}
class Emprestimo {
    +Long id
```

```

+LocalDate dataEmprestimo
+LocalDate dataDevolucaoPrevista
+LocalDate dataDevolucaoEfetiva
}

```

```

"UserDetails" <|.. Usuario : implements

```

```

Usuario "1" -- "0..*" Emprestimo : possui
Livro "1" -- "0..*" Emprestimo : referente_a
Usuario " * " -- " * " Role : tem

```

2.2. Diagrama de Banco de Dados (Entidade-Relacionamento)

Este diagrama mostra as tabelas SQL resultantes do mapeamento JPA, incluindo a tabela de junção usuarios_roles para o relacionamento Muitos-para-Muitos.

erDiagram

```

usuarios {
    BIGINT id PK
    VARCHAR username UK
    VARCHAR password
    VARCHAR nome_completo
}

```

```

roles {
    BIGINT id PK
    VARCHAR nome UK
}

```

```

livros {
    BIGINT id PK
    VARCHAR titulo
    VARCHAR autor
    VARCHAR isbn UK
    INT quantidade_disponivel
}

```

```

emprestimos {

```

```

BIGINT id PK
DATE data_emprestimo
DATE data_devolucao_prevista
DATE data_devolucao_efetiva
BIGINT usuario_id FK
BIGINT livro_id FK
}

usuarios_roles {
    BIGINT usuario_id FK
    BIGINT role_id FK
}

usuarios ||--o{ empréstimos : "possui"
livros ||--o{ empréstimos : "é_emprestado_em"
usuarios }o--o{ roles : "possui_papel_em"

```

3. Padrões de Projeto (GoF) Aplicados

O projeto utiliza diversos padrões de design, tanto do "Gang of Four" (GoF) quanto padrões de arquitetura modernos.

1. Singleton (GoF):

- **Aplicação:** Este padrão é aplicado pelo contêiner de Injeção de Dependência (DI) do Spring.
- **Evidência:** Todas as classes anotadas com `@Service`, `@Repository`, `@Component` e `@Controller` (ex: `LivroService`, `UsuarioService`) são instanciadas como Singletons por padrão. Isso garante que uma única instância desses serviços seja compartilhada por toda a aplicação, gerenciando o estado de forma centralizada e economizando recursos.

2. Factory Method (GoF):

- **Aplicação:** O `TokenService` atua como uma fábrica.
- **Evidência:** O método `gerarToken(Usuario usuario)` é um "método de fábrica". Ele abstrai a complexidade da criação de um objeto complexo (a string JWT assinada e com data de expiração) a partir de um objeto de entrada (o `Usuario`). O `AuthenticationController` não precisa saber *como* um token é construído, apenas *pedir* um.

3. Strategy (GoF):

- **Aplicação:** O `SecurityConfig` (Spring Security) é uma implementação clara do padrão Strategy.

- **Evidência:** Nós definimos múltiplas *estratégias* de autorização para diferentes rotas.
 - requestMatchers("/api/auth/login").permitAll() (Estratégia Pública)
 - requestMatchers("/api/admin/**").hasRole("ADMIN") (Estratégia de Admin)
 - anyRequest().authenticated() (Estratégia de Usuário Logado)
 - O FilterChainProxy do Spring avalia a requisição e seleciona a estratégia de autorização correta em tempo de execução.
4. **Facade (GoF):**
- **Aplicação:** O DashboardService atua como uma Fachada.
 - **Evidência:** O AdminController precisa de estatísticas gerais (getAdminStats()). Em vez de o controller chamar UsuarioRepository.count(), LivroRepository.count() e EmprestimoRepository.countBy...(), ele faz uma única chamada ao DashboardService. O serviço atua como uma fachada, escondendo a complexidade de coordenar múltiplos repositórios para entregar um DTO simples (AdminDashboardStatsDTO).
5. **Data Transfer Object (DTO) (Padrão de Arquitetura):**
- **Aplicação:** Usado extensivamente em toda a API.
 - **Evidência:** LivroDTO, LivroComStatusDTO, LoginDTO, AdminUsuarioCreatedDTO, etc. Esses objetos separam o modelo de persistência (Entidade) do modelo da API (DTO), garantindo que dados sensíveis (como password) nunca sejam expostos e que a API possa evoluir sem quebrar o banco de dados.

4. Evidências de Boas Práticas (Clean Code & SOLID)

4.1. Princípios SOLID

- **S - Princípio da Responsabilidade Única (SRP):**
 - **Evidência:** Nossa arquitetura em camadas é a maior prova.
 - LivroController só lida com HTTP e JSON.
 - LivroService só lida com regras de negócio (ex: verificar estoque, formatar DTOs).
 - LivroRepository só lida com SQL (via JPA).
 - GlobalExceptionHandler só lida com tratamento de erros.
 - No frontend, o useEffect de busca só busca dados, e o handle...Submit só lida com o envio de formulários.
- **O - Princípio Aberto/Fechado (OCP):**
 - **Evidência:** O SecurityConfig. Quando precisamos adicionar uma nova funcionalidade (ex: ProfileController), nós *adicionamos* uma nova regra de segurança (.requestMatchers("/api/profile/**").authenticated()) sem *modificar* o

SecurityFilter existente, que já funciona. A lógica está aberta para extensão, mas fechada para modificação.

- **L - Princípio da Substituição de Liskov (LSP):**

- **Evidência:** A integração Usuario <-> UserDetails. O Spring Security foi programado para a interface UserDetails. Nossa classe Usuario implementa UserDetails e é "substituída" no lugar onde o Spring espera a interface. O AuthenticationManager chama usuario.getAuthorities() sem saber (ou se importar) que é a nossa classe customizada, e o sistema funciona perfeitamente.

- **I - Princípio da Segregação de Interfaces (ISP):**

- **Evidência:** O uso de DTOs específicos. O frontend não recebe a entidade Usuario inteira (que tem dezenas de métodos de UserDetails). Ele recebe um LoginResponseDTO que contém *apenas* o que ele precisa: token, nomeCompleto e roles. Criamos interfaces (DTOs) segregadas e específicas para cada caso de uso.

- **D - Princípio da Inversão de Dependência (DIP):**

- **Evidência:** Injeção de Dependência do Spring (@Autowired). Nossas classes de alto nível (Services) não dependem de implementações concretas, mas sim de abstrações (Interfaces).
- Ex: LivroService depende da *interface* LivroRepository, não de uma classe concreta. O Spring "inverte o controle" e nos *fornece* a implementação em tempo de execução.

4.2. Práticas de Clean Code

- **Nomes Significativos:**

- **Frontend:** Funções como handleEmprestarLivro, handleLogout, handleSubmitLogin. Variáveis como isAdmin, editingBook.
- **Backend:** Métodos como realizarEmprestimo, deletarUsuario. Classes como EmprestimoRepository, PasswordChangeDTO.

- **Funções Pequenas e Focadas:**

- A maioria dos métodos de controller tem 1 ou 2 linhas: receber a requisição, chamar o serviço e retornar a resposta.
- No React, handleLogout só faz logoff, handleSearchChange só atualiza o termo.

- **Tratamento de Erros Centralizado:**

- **Backend:** O GlobalExceptionHandler intercepta exceções (RuntimeException, MethodArgumentNotValidException) e as transforma em respostas JSON padronizadas. Isso evita blocos try-catch poluindo todos os métodos do controller.

- **Frontend:** A lógica de catch (err) { ... } em todas as chamadas axios atualiza os *states* de erro (setAuthError, setSearchError, setGeneralStatus), exibindo feedback claro para o usuário sem "quebrar" a aplicação.
- **Segurança (Backend):**
 - **Validação de Negócio:** O backend *nunca* confia no frontend. O EmprestimoService (e não o React) é quem verifica o estoque e o limite do usuário.
 - **Hashing de Senhas:** O PasswordEncoder (BCrypt) é usado em todos os locais onde senhas são salvas ou verificadas.
 - **Proteção de API:** O backend, não o frontend, é quem decide quem é Admin (@AuthenticationPrincipal e hasRole("ADMIN")).